

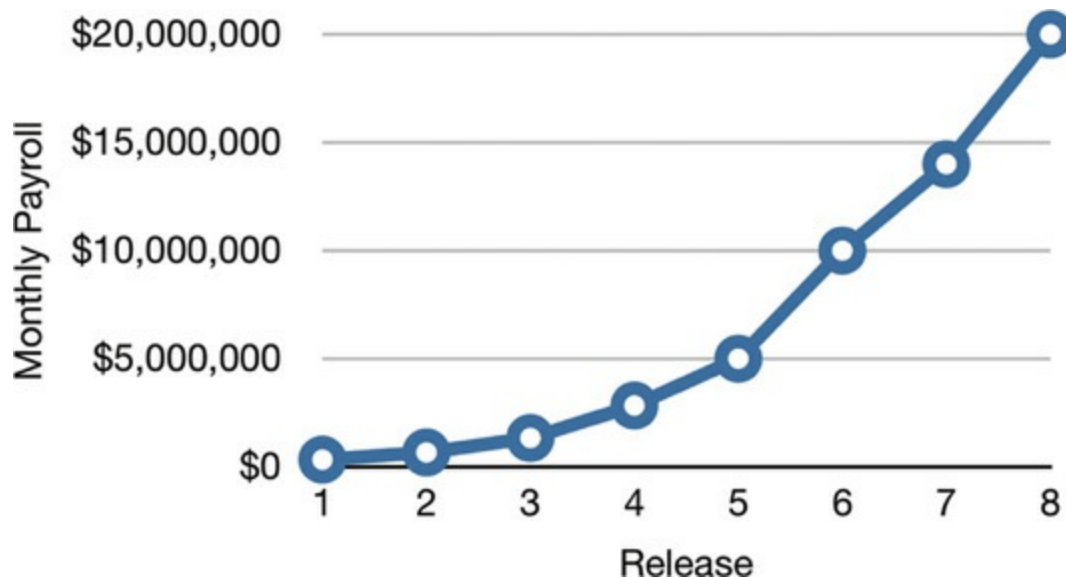


Figure 1.5 Monthly development payroll by release

Release 1 was delivered with a monthly payroll of a few hundred thousand dollars. The second release cost a few hundred thousand more. By the eighth release monthly payroll was \$20 million, and climbing.

Just this chart alone is scary. Clearly something startling is happening. One hopes that revenues are outpacing costs and therefore justifying the expense. But no matter how you look at this curve, it's cause for concern.

But now compare the curve in [Figure 1.5](#) with the lines of code written per release in [Figure 1.2](#). That initial few hundred thousand dollars per month bought a lot of functionality—but the final \$20 million bought almost nothing! Any CFO would look at these two graphs and know that immediate action is necessary to stave off disaster.

But which action can be taken? What has gone wrong? What has caused this incredible decline in productivity? What can executives do, other than to stamp their feet and rage at the developers?

WHAT WENT WRONG?

Nearly 2600 years ago, Aesop told the story of the Tortoise and the Hare. The moral of that story has been stated many times in many different ways:

- “Slow and steady wins the race.”
- “The race is not to the swift, nor the battle to the strong.”

- “The more haste, the less speed.”

The story itself illustrates the foolishness of overconfidence. The Hare, so confident in its intrinsic speed, does not take the race seriously, and so naps while the Tortoise crosses the finish line.

Modern developers are in a similar race, and exhibit a similar overconfidence. Oh, they don’t sleep—far from it. Most modern developers work their butts off. But a part of their brain *does* sleep—the part that knows that good, clean, well-designed code *matters*.

These developers buy into a familiar lie: “We can clean it up later; we just have to get to market first!” Of course, things never do get cleaned up later, because market pressures never abate. Getting to market first simply means that you’ve now got a horde of competitors on your tail, and you have to stay ahead of them by running as fast as you can.

And so the developers never switch modes. They can’t go back and clean things up because they’ve got to get the next feature done, and the next, and the next, and the next. And so the mess builds, and productivity continues its asymptotic approach toward zero.

Just as the Hare was overconfident in its speed, so the developers are overconfident in their ability to remain productive. But the creeping mess of code that saps their productivity never sleeps and never relents. If given its way, it will reduce productivity to zero in a matter of months.

The bigger lie that developers buy into is the notion that writing messy code makes them go fast in the short term, and just slows them down in the long term. Developers who accept this lie exhibit the hare’s overconfidence in their ability to switch modes from making messes to cleaning up messes sometime in the future, but they also make a simple error of fact. The fact is that *making messes is always slower than staying clean*, no matter which time scale you are using.

Consider the results of a remarkable experiment performed by Jason Gorman depicted in [Figure 1.6](#). Jason conducted this test over a period of six days. Each day he completed a simple program to convert integers into Roman numerals. He knew his work was complete when his predefined set of acceptance tests passed. Each day the task took a little less than 30 minutes. Jason used a well-known cleanliness discipline named test-driven development (TDD) on the first, third, and fifth days. On the other three days, he wrote the code without that discipline.